Day 108: Feature Importance — Permutation Against Impurity-Based

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 4 | Complete study guide

Watch: 13.4.2 Feature Permutation Importance (Sebastian Raschka) | 5.5 Permutation Feature Importance (KHU IAILab)

Script: `day108_feature_importance.py` · every number in this guide is copied by program from `day108_run_output.txt`, the real output of that script.

Two popular ways to ask a forest what mattered, written from scratch and tested where the right answer is known. Neither can be trusted alone. Impurity importance ranks `PassengerId`, a row number, third of eight (0.1910, above Age), while permutation on test rows gives it 0.0045. Permutation has the opposite failure: on breast cancer it says 26 of 30 features cost nothing to shuffle, yet shuffling one correlated group together costs 0.2813. So the two methods disagree most exactly where the data is hardest to read.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

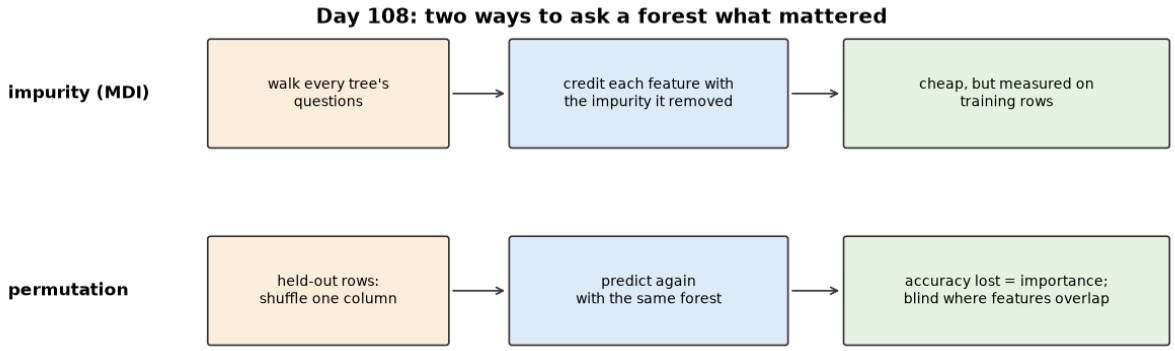
Feature importance gives each input a score for how much a trained model relies on it. A score describes the *model*, not the world: a feature can matter to the model and cause nothing.

Impurity importance (MDI, mean decrease in impurity) adds up how much impurity each feature removed during training. **Permutation importance** shuffles one column of held-out data and measures the accuracy lost. **Drop-column importance** retrains without the feature: slow, and used here as the reference.

REAL-WORLD SCENARIO

To judge a player's value, you can count their goals (impurity), bench them in tomorrow's match and watch the score (permutation), or rebuild the whole season without them (drop-column). Each answer is cheaper than the next, and less complete.

SYSTEM DIAGRAM



The two procedures: impurity reads the trees; permutation breaks a column and predicts again.

Today's part for Day 111

`day108_exercise.py` is both measures, with permutation written for any model that has a score function. Its real output:

ACTUAL OUTPUT

```
by hand: 0.3 / 0.34 = 0.8824, 0.04 / 0.34 = 0.1176; tree_importance: [0.8824, 0.1176, 0.0]
two identical trees average to the same shares: [0.8824, 0.1176, 0.0]
a perfect rule on column 0: importance [0.4984, 0.0, 0.0] (about 0.5, because a shuffled column is
    right half the time)
spearman: same order 1.0; reversed -1.0
```

Data: breast cancer and digits (bundled with scikit-learn) and Titanic, the repository's own copy. The forest is Day 107's. scikit-learn appears only as an outside check.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Correctness checks

DEFINITION — start here, no prior knowledge assumed

At each question, the feature used is credited with the impurity removed, weighted by the share of rows that reached the question. Each tree's totals become shares that sum to 1, and the forest averages them.

REAL-WORLD SCENARIO

Crediting each salesperson by the size of the sales they closed, then reporting shares of the total.

MATHEMATICS

$$MDI(f) = \text{average over trees of } \Sigma_{\text{questions using } f} (n_{\text{node}} / n) \times \text{gain, as a share of the tree's total}$$

$$PI(f) = \text{score}(X) - \text{average over shuffles of score}(X \text{ with column } f \text{ shuffled})$$

IMPLEMENTATION

```
def tree_importance(tree, p):
    out = np.zeros(p); stack = [tree]; n = tree["counts"].sum()
    while stack:
        node = stack.pop()
        if "feature" in node:
            out[node["feature"]] += node["counts"].sum() / n * node["gain"]
            stack += [node["left"], node["right"]]
    return out / max(out.sum(), 1e-12)
def impurity_importance(trees, p):
    return np.mean([tree_importance(t, p) for t in trees], axis=0)
def permutation_importance(trees, X, y, k, seed, groups=None):
    rng = np.random.RandomState(seed); base = accuracy(forest_proba(trees, X, k), y)
    groups = groups if groups is not None else [[j] for j in range(X.shape[1])]
    out = np.zeros((len(groups), REPEATS))
    for g, cols in enumerate(groups):
        for r in range(REPEATS):
            Z = X.copy(); Z[:, cols] = X[rng.permutation(len(X))][:, cols]
            out[g, r] = base - accuracy(forest_proba(trees, Z, k), y)
    return out.mean(1), out.std(1)
```

ACTUAL OUTPUT

```
by hand: feature 0 removes 0.3 at the root (20 of 20 rows), feature 1 removes 0.1 on 8 of 20 rows: 0.3 / 0.34
    = 0.8824, 0.04 / 0.34 = 0.1176
tree_importance gives [0.8824, 0.1176, 0.0] (the third feature was never used)
one tree, breast cancer: largest gap to scikit-learn's feature_importances_ 0.0154 (tied splits may differ)
forest of 300: rank correlation with scikit-learn's forest 0.957; largest gap 0.0208
Actual output from this lesson's run.
```

What it means

The hand calculation matches (0.8824, 0.1176), and an unused feature scores exactly 0. On one real tree the largest gap to scikit-learn is 0.0154, because tied splits may differ. Two different forests agree closely in ranking (0.957), though neither forest is the 'true' one.

WHY CHOOSE THIS?

Both are short enough to check by hand.

WHY NOT THIS? (TRADEOFFS)

A check that the numbers are computed correctly says nothing yet about whether they mean what we want.

2. Breast cancer: the two rankings side by side

DEFINITION — start here, no prior knowledge assumed

One forest, two rankings. Impurity importance is read from the training; permutation is measured on 171 test tumours with 10 shuffles per feature.

REAL-WORLD SCENARIO

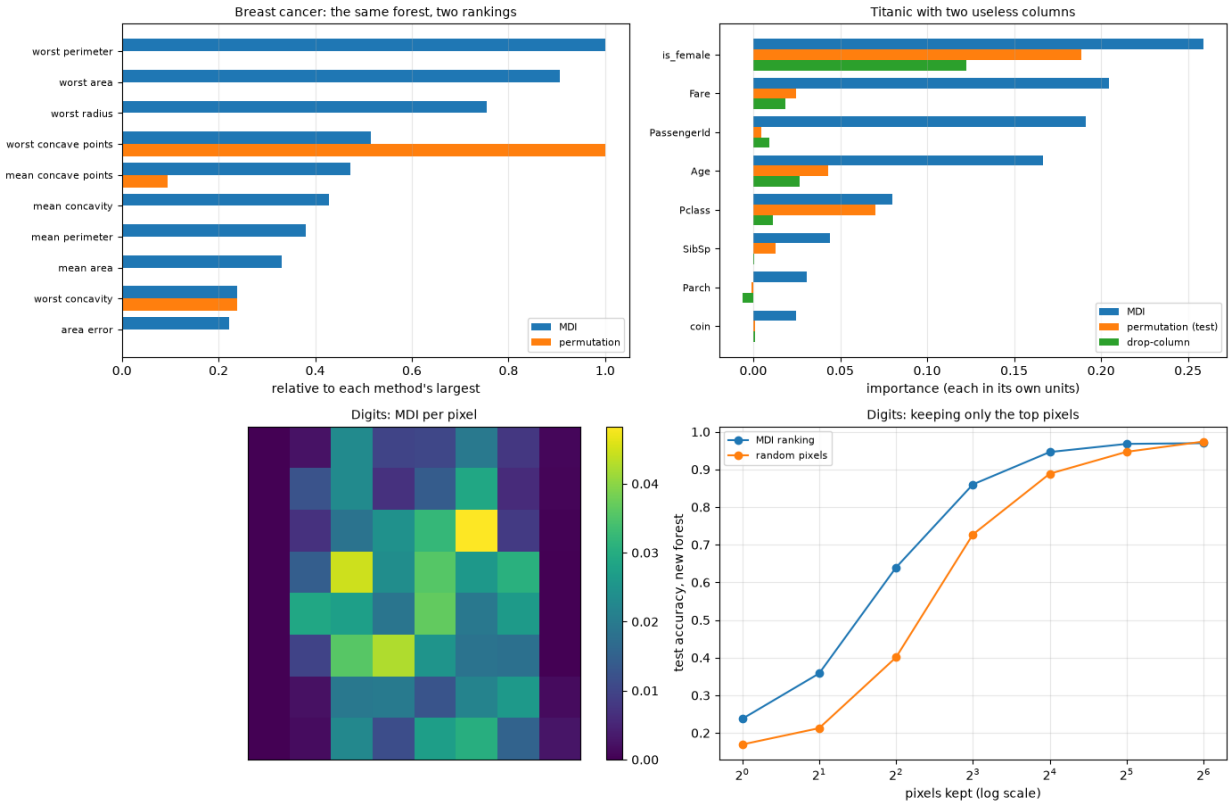
Two referees rate the same player very differently, and you need to know why before believing either.

SPECIFICATION

Forest	100 trees, mtry 5, trained on 398 tumours.
Permutation	Accuracy lost on the 171 test tumours, 10 shuffles per feature. Negative means shuffling happened to help.

ACTUAL OUTPUT

```
forest of 100, test accuracy 0.9708; permutation on the 171 test tumours, 10 shuffles each
feature                     MDI  MDI rank     perm      +-  perm rank
worst perimeter          0.1481         1  -0.0070  0.0051         30
worst area               0.1341         2  -0.0053  0.0031         28
worst radius             0.1118         3  -0.0058  0.0045         29
worst concave points     0.0763         4   0.0123  0.0049          1
mean concave points      0.0699         5   0.0012  0.0044          4
mean concavity           0.0636         6  -0.0006  0.0041         25
mean perimeter           0.0563         7   0.0000  0.0000         22
mean area                0.0489         8   0.0000  0.0000         21
worst concavity          0.0352         9   0.0029  0.0029          2
area error               0.0328        10  -0.0018  0.0046         26
rank correlation of the two lists: -0.280; features whose shuffle cost no accuracy at all: 26 of 30
largest single loss 0.0123, which is 2.1 tumours of 171, though the features share a lot of information
Actual output from this lesson's run.
```



Top left: the two rankings. Top right: Titanic with useless columns. Bottom left: importance per pixel on digits. Bottom right: accuracy with only the top pixels.

What it means

They disagree badly: rank correlation -0.280. MDI's top three, the worst perimeter, area and radius, come bottom on permutation, at ranks 30, 28 and 29. The largest single loss is only 2.1 tumours. It would be easy to conclude that size is irrelevant to a tumour's diagnosis, and that would be wrong. Section 3 shows why.

WHY CHOOSE THIS?

Comparing two measures shows you when to stop trusting either.

WHY NOT THIS? (TRADEOFFS)

One split and 171 rows: the permutation column is noisy, and 0.0123 is about two tumours.

3. Correlated features

DEFINITION — start here, no prior knowledge assumed

Radius, perimeter and area measure almost the same thing. Shuffle one, and the trees simply use the others, so each looks useless. The fix is to **shuffle the whole group together**, so no near-copy can step in.

REAL-WORLD SCENARIO

Three people hold the same spare key. Removing any one of them locks nobody out; removing all three does.

IMPLEMENTATION

```
def clusters(X, limit=0.9):
    r = np.abs(np.corrcoef(X.T)); left = list(range(X.shape[1])); out = []
    while left:
        j = left.pop(0); g = [j] + [i for i in left if r[j, i] > limit]
        left = [i for i in left if i not in g]; out.append(g)
    return out
```

ACTUAL OUTPUT

```
greedy groups with |r| > 0.9 on the training rows: 21 groups from 30 features, 4 of them with more than one
    member
largest groups                                          features best single  together
mean radius, mean perimeter, mean area, worst radi...          6      0.0000    0.2813
worst concave points                                           1      0.0123    0.0175
mean concavity, mean concave points                            2      0.0012    0.0164
mean texture, worst texture                                    2      0.0012    0.0070
worst concavity                                                1      0.0029    0.0018
Actual output from this lesson's run.
```

What it means

The size group of 6 features has no single member worth anything (0.0000), yet shuffling it together costs 0.2813: the largest effect in this lesson. Permutation importance answers 'what happens if this *one* column breaks?', not 'does this information matter?'. Group by correlation first when features overlap.

WHY CHOOSE THIS?

Group shuffling is one line more, and answers the real question.

WHY NOT THIS? (TRADEOFFS)

The |r| limit of 0.9 is a choice, and only sees straight-line relations.

4. Useless columns: a real ID and a random coin

DEFINITION — start here, no prior knowledge assumed

Titanic gains two columns that cannot matter: `PassengerId`, a row number with a different value for every passenger, and a fair coin. MDI is known to favour columns with many distinct values, since they offer many chances for a split to look good by luck (Strobl et al., 2007).

REAL-WORLD SCENARIO

A horoscope tried with 891 different wordings will fit somebody's life. Tried with two wordings, it rarely does.

SPECIFICATION

Forest	100 trees, mtry 3 of 8, 10 splits.
Columns	MDI; permutation on training rows; permutation on test rows; drop-column (retrain without it).
Note	Each column is in its own units. Compare within a column, not across.

ACTUAL OUTPUT

```
10 splits, 100 trees, mtry 3 of 8; permutation on training and on test rows; drop-column retrains without the
    feature
feature       distinct     MDI  perm train  perm test  drop-column   +- SE
is_female            2  0.2589      0.2763     0.1887      +0.1224  0.0056
Fare               248  0.2043      0.1307     0.0247      +0.0183  0.0047
PassengerId        891  0.1910      0.1028     0.0045      +0.0093  0.0055
Age                 88  0.1666      0.1221     0.0433      +0.0265  0.0059
Pclass               3  0.0798      0.1322     0.0701      +0.0112  0.0049
SibSp                7  0.0439      0.0286     0.0129      +0.0007  0.0043
Parch                7  0.0307      0.0170    -0.0011      -0.0063  0.0057
coin                 2  0.0247      0.0378     0.0008      +0.0011  0.0049
Actual output from this lesson's run.
```

What it means

MDI puts a row number third, at 0.1910, above Age (0.1666), and Pclass, with only 3 values and real signal, far below (0.0798). The trees really did split on it: they fit noise, and MDI reports that as importance. Permutation on *training* rows falls for the same trap (0.1028), because the forest has memorised those rows. Only held-out rows expose it (0.0045). Drop-column gives +0.0093 with a standard error of 0.0055: under two standard errors from zero, on a column with no meaning. Treat that as the resolution of the method on 10 splits. Note too that Pclass loses 0.0701 when shuffled but only +0.0112 when dropped, perhaps because a retrained forest can lean on Fare, which overlaps with class (not tested here).

WHY CHOOSE THIS?

Held-out permutation is honest about noise columns, and model-agnostic.

WHY NOT THIS? (TRADEOFFS)

It needs held-out rows and many predictions; drop-column is slower still.

5. A duplicated feature

DEFINITION — start here, no prior knowledge assumed

An exact copy of `is_female` is added. A good measure might give each half, or both the full credit. Which one happens?

REAL-WORLD SCENARIO

Two identical witnesses: remove either and the case stands.

ACTUAL OUTPUT

```
10 splits, 100 trees, mtry 2
                    MDI  perm test
alone            0.2685     0.1757
with a copy      0.1427     0.0613
the copy         0.1441     0.0593
the two added    0.2868     0.1206
Actual output from this lesson's run.
```

What it means

MDI divides the credit: 0.2685 becomes 0.1427 and 0.1441, adding up to a little more. Permutation loses signal: 0.1757 becomes 0.0613 and 0.0593, which add up to only 0.1206. So one truly important fact, stored twice, can look unimportant on both measures. Real tables contain near-copies all the time.

WHY CHOOSE THIS?

A clean test: the true answer is known by construction.

WHY NOT THIS? (TRADEOFFS)

An exact copy is the extreme case; section 3 shows the usual one.

6. Stability

DEFINITION — start here, no prior knowledge assumed

The same procedure on 10 splits. A **rank correlation** near 1 says two runs order the features alike. Top-5 overlap asks the practical question: would you get the same shortlist?

REAL-WORLD SCENARIO

A bathroom scale that gives the same reading every morning is reliable, but it may still be wrong.

MATHEMATICS

$$Spearman(a, b) = Pearson\ correlation\ of\ the\ ranks\ of\ a\ and\ the\ ranks\ of\ b$$

ACTUAL OUTPUT

```
MDI         : rank correlation between splits 0.930; top-5 lists share 4.42 of 5 features on
    average
permutation : rank correlation between splits 0.144; top-5 lists share 1.89 of 5 features on
    average
Actual output from this lesson's run.
```

What it means

MDI is stable (0.930, and 4.42 of 5 shared); permutation is not (0.144, 1.89 of 5). That is partly because permutation here is measured on 171 rows, where a change of one tumour is large. And stability is not correctness: a measure can repeat the same wrong answer, and section 4 showed MDI's answer about a row number was wrong.

WHY CHOOSE THIS?

Tells you how far to trust one printed ranking.

WHY NOT THIS? (TRADEOFFS)

These splits share rows, so true stability on new data is likely lower.

7. Using it: how few pixels does a forest need?

DEFINITION — start here, no prior knowledge assumed

Digits has 64 pixels. Rank them by MDI *on training rows only*, keep the top k, and train a new forest. A random choice of k pixels is the baseline.

REAL-WORLD SCENARIO

Packing for a trip: the first few items matter a lot, and the rest hardly at all.

ACTUAL OUTPUT

```
1797 images of 8x8 pixels, 10 classes; 3 splits; 50 trees; every ranking made on training rows only, then a
    new forest is trained
 pixels kept       1       2       4       8      16      32      64
MDI ranking   0.2375  0.3581  0.6395  0.8602  0.9462  0.9678  0.9697
random pixels 0.1694  0.2127  0.4007  0.7273  0.8887  0.9468  0.9740
pixels never used by any tree: 4 of 64; those pixels are non-blank in 0.0006 of images, the others in 0.5447
Actual output from this lesson's run.
```

What it means

Eight chosen pixels give 0.8602 against 0.7273 for eight random ones, and sixteen give 0.9462, close to all sixty-four (0.9697). At 64 the two rows hold the same pixels, and differ only in column order: 0.9697 against 0.9740. That gap is a useful measure of the noise here. Here MDI has a clear physical reading too: the forest uses the middle of the image. 4 pixels are never used, and they are non-blank in only 0.0006 of images.

WHY CHOOSE THIS?

Fewer features can mean cheaper, faster models.

WHY NOT THIS? (TRADEOFFS)

Choosing by accuracy on the test set would leak; here the ranking uses training rows only.

Interview preparation — technical questions

Q: How is impurity importance computed?

A: Sum each question's gain, weighted by the share of rows reaching it, per feature; normalise per tree; average over the forest.

Q: How is permutation importance computed?

A: Shuffle one column of held-out data, predict again, and record the score lost; repeat and average.

Q: Why on held-out rows?

A: On training rows an overfitted forest looks as if it depends on noise: PassengerId 0.1028 on training, 0.0045 on test.

Q: How do you handle correlated features?

A: Group them and shuffle the group together: here 0.0000 for the best single member and 0.2813 for the group.

Q: What does a negative permutation importance mean?

A: Shuffling happened to help, so the true effect is about zero. It is noise.

Interview preparation — theoretical questions

Q: Why does MDI favour high-cardinality features?

A: Many distinct values give many thresholds, so some split looks good by chance. It is measured on the training data, so it cannot tell.

Q: Does an important feature cause the outcome?

A: No. Importance describes the model. PassengerId scored 0.1910 on MDI and causes nothing.

Q: Is permutation model-agnostic?

A: Yes: it needs only predictions, so Day 111 can use it for a neural network too.

Q: Why might both methods mislead you on real data?

A: MDI credits noise; permutation hides information that is shared among correlated features. Using both, and retraining, shows more.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Importance is a measurement with known failure modes, not a fact. Impurity importance is stable, cheap and fooled by noise columns. Permutation on held-out rows sees through noise but goes blind when features overlap, unless they are grouped. Neither says anything about cause. Report which one you used, on which rows, and how much it changes between runs.

- Breast cancer: rank correlation between the two measures -0.280; a shuffled size group costs 0.2813.
- Titanic: PassengerId ranks third on MDI (0.1910), and 0.0045 on held-out permutation.
- A duplicate splits MDI (0.1427 + 0.1441) and shrinks permutation (0.0613 + 0.0593).
- Across splits, MDI's rank correlation is 0.930 and permutation's 0.144.
- Digits: 16 MDI-chosen pixels of 64 give 0.9462 against 0.9697 for all.

Suggested watch order, if starting from zero

- 1. 13.4.2 Feature Permutation Importance (Sebastian Raschka) — permutation importance step by step.
- 2. 5.5 Permutation Feature Importance (KHU IAILab) — a second lecture on the method.

Day 109 preview

- **Gradient boosting from scratch.** Trees grown one after another, each correcting the errors of those before it.